

1. **The DOM tree:** What the DOM is · Selecting elements: `getElementById`, `querySelector`, `querySelectorAll` · Loading scripts: `defer` and `DOMContentLoaded`
2. **Events:** Events and `addEventListener` · The event object: `target` and `currentTarget` · `preventDefault`: forms and links
3. **Changing the page:** Creating, adding and removing elements · `textContent` vs `innerHTML`, and the XSS risk · Attributes, properties and form values · ARIA attributes and keyboard focus · `classList`: styling through classes
4. **Propagation and delegation:** Event propagation: capture, target and bubble · `stopPropagation`, and when not to use it · Event delegation
5. **Putting it together:** State → render: a small to-do pattern

The DOM tree

What the DOM is

The **DOM** (Document Object Model) is the browser's live, in-memory tree of objects built from your HTML; JavaScript reads and changes the page through it.



The screen shows the DOM, not the file; JavaScript edits the DOM.

Element nodes Come from tags: `<li>` becomes an element node.

Text nodes Hold the text inside elements, including the spaces and line breaks between tags.

document The entry point to the tree: `document.body` is the `<body>` element.

Moving around the tree

Property	Gives
<code>el.parentElement</code>	The parent element
<code>el.children</code>	The child elements only
<code>el.firstChild</code> , <code>el.nextElementSibling</code>	The first child element; the next sibling element
<code>el.firstChild</code> , <code>el.childNodes</code>	Text nodes too: often a whitespace text node

Example. You write `<table><tr><td>1</td></tr></table>`. In the DOM the row sits inside a `<tbody>` the browser added, so `document.querySelector('table > tr')` finds nothing, while `document.querySelector('table tr')` (any depth) works.

Common mistake. Looking for elements created by JavaScript in **View Source** and concluding the code failed. View Source shows the original file; open the **Elements** panel to see the live DOM.

Selecting elements: `getElementById`, `querySelector`, `querySelectorAll`

`document.getElementById(id)` **and** `document.querySelector(css)` **return one element (or null);**
`document.querySelectorAll(css)` **returns every match as a NodeList.**

The three selection methods

Call	Argument	Returns	No match
<code>document.getElementById('save')</code>	An id, without #	One element	<code>null</code>
<code>document.querySelector('#save')</code>	Any CSS selector	The first matching element	<code>null</code>
<code>document.querySelectorAll('.item')</code>	Any CSS selector	A static <code>NodeList</code>	An empty <code>NodeList</code> (length 0)

- **A `NodeList` is array-like:** it has `length`, `list[0]`, `forEach` and works with `for...of`, but has no `map` or `filter`. Convert it with `Array.from(nodes)` or `[...nodes]`.
- **A snapshot:** elements added to the page later are not in it.

- **Search inside an element:** `menu.querySelector('a')` looks only among the descendants of `menu`.

Example. In a task list, `document.querySelectorAll('.tasks li.done')` finds the completed tasks, `document.querySelector('.tasks li')` the first task, and `document.querySelector('.tasks li:last-child')` the last one: the same selectors you would write in a stylesheet.

Common mistake. Passing a CSS selector to `getElementById`: `document.getElementById('#save')` looks for an element whose id is literally `#save`, returns `null`, and the next line fails with `TypeError: Cannot read properties of null (reading 'addEventListener')`.

Loading scripts: defer and DOMContentLoaded

A script can only find elements the browser has already parsed; `<script src="app.js" defer>` (or a script at the end of `<body>`) runs after the whole HTML has been read.

Where the script tag goes and what it can see

Script tag	Runs	Finds <code><body></code> elements?
<code><script src="app.js"></code> in <code><head></code>	At once, blocking the parsing	No
<code><script src="app.js" defer></code> in <code><head></code>	After parsing, in order	Yes (recommended)
<code><script src="app.js"></code> just before <code></body></code>	When the parser reaches it, almost at the end	Yes, the elements above it
<code><script src="app.js" async></code>	As soon as it is downloaded	Not reliably

- **defer is the default to reach for:** the script downloads in parallel and runs after the HTML is parsed, in the order the scripts appear. Scripts with `type="module"` are deferred automatically.
- `DOMContentLoaded` is an event that fires on document when the HTML has been parsed (it does not wait for images; `load` on window does). Listening for it (see [Events and addEventListener](#)) is the older way to the same guarantee.
- `async` scripts run as soon as they arrive, in any order: fine for independent scripts such as analytics, wrong for code that needs the page.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>To-do</title>
  <link rel="stylesheet" href="styles.css">
  <script src="app.js" defer></script>
</head>
<body>
  <ul id="list"></ul>
</body>
</html>
```

Example. In the Try it box, the first inline script runs before the paragraph exists and finds nothing; the second, placed after it, finds it; the `DOMContentLoaded` listener runs last, once everything has been parsed.

Common mistake. Moving the script into `<head>` without `defer`. The page shows `TypeError: Cannot read properties of null (reading 'addEventListener')`, not because the selector is wrong but because the element did not exist yet when the line ran.

Events

Events and addEventListener

An event is the browser's signal that something happened (a click, a key, a form submission);

`el.addEventListener(type, handler)` **registers a function the browser calls every time that event happens on that**

element.

Your script registers and ends; the browser calls your handler on every click.

Events you will use most

Event	Fires when	Typical target
click	The element is clicked, or activated with Enter or Space on a button	Buttons, links, list items
input	The value of a field changes, on every keystroke	<input>, <textarea>
change	The user commits a change: leaves the field, ticks a box, picks an option	Checkboxes, <select>
submit	A form is submitted: a button click or Enter in a text field	<form>
keydown	A key is pressed; e.key says which ('Enter', 'Escape', 'a')	Fields, document
DOMContentLoaded	The HTML has been fully parsed	document

- **Pass the function, do not call it:** `addEventListener('click', save)`, or an arrow such as `() => save(42)`.
- **Many listeners** for the same event run in the order they were added.
- `removeEventListener` needs the very same function reference, so only named functions can be removed.

Example. A live character counter: `bio.addEventListener('input', () => { counter.textContent = bio.value.length + ' / 160'; });`. The handler runs on every keystroke and always reads the current value.

Common mistake. Writing `btn.addEventListener('click', handleClick());` with parentheses. `handleClick` runs once, immediately, and its return value (usually `undefined`) is registered as the "handler", so clicks then do nothing. The older `onclick="..."` HTML attribute is best avoided too: it mixes JavaScript into the markup and allows only one handler.

The event object: target and currentTarget

Every handler receives an event object describing what happened: `e.type`, `e.target` (where the event started), `e.currentTarget` (whose listener is running), `e.key` for keys, and methods such as `preventDefault()`.

- `e.target`** The innermost element where the event started: the thing actually clicked.
- `e.currentTarget`** The element the listener was added to.
- `e.type`, `e.key`** Which event ('click'), and for key events which key ('Enter').
- `e.target.value`** For input events, the field's current text.
- `e.preventDefault()`** Cancels the browser's own reaction (see [preventDefault](#)).

- `e.target.closest(selector)` climbs from the target through its ancestors (itself included) and returns the first match, or `null`: the safe way to ask "which item was this click inside?".
- **Arrow functions:** inside an arrow handler, `this` is not the element; use `e.currentTarget`. (In a regular function handler, `this` is `e.currentTarget`.)

Example. A button with an icon: `<button data-id="7"> Delete</button>`. Clicking the icon gives `e.target = the <img>`, which has no `data-id`. `e.currentTarget.dataset.id` (listener on the button) or `e.target.closest('button').dataset.id` gives '7' wherever the user clicked.

Common mistake. Using `e.target` when you meant "the element I attached the listener to". It works while you click on bare text and breaks as soon as the element contains children (an icon, a `<strong>`, a `<span>`).

preventDefault: forms and links

Some events have a built-in browser reaction: submitting a form sends it and reloads the page, clicking a link

navigates. `e.preventDefault()` **cancels that default action so your JavaScript can handle the event instead.**

- **Listen for submit on the** `<form>`, not for click on its button: submit fires for a button click **and** for Enter in a text field, and only after HTML validation such as required has passed.
- **Buttons in a form submit** by default (`type="submit"`); give other buttons `type="button"`.
- **Only the default is cancelled:** the event still bubbles to the ancestors. Stopping that is stopPropagation, a different tool.

```
<form id="todo-form">
  <label for="todo-input">New task</label>
  <input id="todo-input" required>
  <button>Add</button>      <!-- type="submit" by default -->
</form>

<script>
const form = document.querySelector('#todo-form');
const input = document.querySelector('#todo-input');

form.addEventListener('submit', (e) => {
  e.preventDefault();          // no reload: we handle it
  const text = input.value.trim();
  if (!text) return;
  console.log('add', text);
  form.reset();                // empty the field
  input.focus();
});
</script>
```

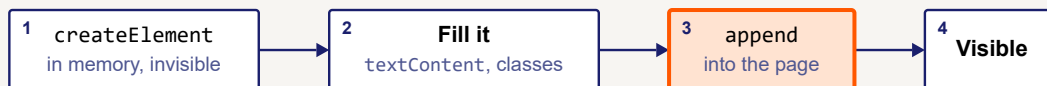
Example. The classic to-do form handler: `form.addEventListener("submit", (e) => { e.preventDefault(); ... })`. Remove the `preventDefault()` line and the page reloads on every submit, so the `<li>` you just appended is gone. (The Try it box cannot submit forms, so it shows the same idea on a link and a checkbox.)

Common mistake. Attaching a click listener to the submit button and forgetting `preventDefault()`: the item is added, the form submits, the page reloads, and the item vanishes. The button's click also fires before HTML validation, whereas submit waits for it.

Changing the page

Creating, adding and removing elements

`document.createElement(tag)` **makes a new element that is not on the page yet;** `append` **attaches it to a parent so it appears,** and `el.remove()` **takes it out again.**



Create, fill, attach: nothing shows until the element is attached.

Adding and removing

Call	Does
<code>parent.append(a, b, 'text')</code>	Adds one or more nodes or strings at the end
<code>parent.appendChild(node)</code>	Adds one node at the end: the classic form most tutorials use
<code>parent.prepend(x), el.before(x), el.after(x)</code>	Adds at the start, or next to an element
<code>el.remove()</code>	Takes the element out (older equivalent: <code>parent.removeChild(el)</code>)
<code>list.replaceChildren()</code>	Empties the element in one go

```
const names = ['Ana', 'Bea', 'Carlos'];
const list = document.querySelector('#people');
```

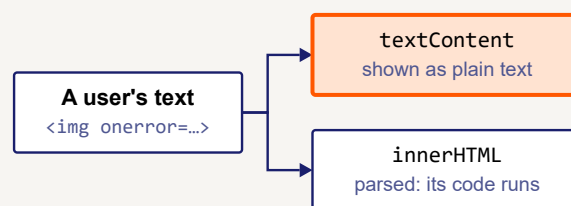
```
for (const name of names) {
  const li = document.createElement('li'); // create
  li.textContent = name;                  // fill (safe: plain text)
  li.classList.add('person');
  list.append(li);                        // attach: now visible
}
```

Example. Rendering an array: for each name, create an `<li>`, set its `textContent`, append it to the `<ul>`. Three names produce three list items; an empty array produces an empty list, with no special case needed.

Common mistake. Creating and filling an element but never attaching it. There is no error at all; the element simply never appears. `el.isConnected` tells you whether it is in the page.

textContent vs innerHTML, and the XSS risk

`textContent` **reads or writes plain text**; `innerHTML` **reads or writes HTML that the browser parses into elements**, so **putting untrusted text into `innerHTML` can run an attacker's code (XSS)**.



The same string: shown as text, or turned into elements that can run code.

- **Data from users or APIs goes into `textContent`**, or into elements built with `createElement`.
- `innerHTML` **only for fixed markup you wrote yourself**. `el.innerHTML = ''` (or `el.replaceChildren()`) is fine to empty an element.
- `innerHTML +=` **rebuilds every child**: the listeners on them are lost and typed input is reset.
- `innerText` depends on CSS (it skips hidden text) and is slower; prefer `textContent`.

Example. A comment box. A visitor posts `<img src=x onerror="stealCookies()">`. With `comment.innerHTML = text` every reader of the page runs `stealCookies()`; with `comment.textContent = text` readers just see that odd text. In the Try it box the "attack" only logs a message, but it proves the code ran.

Common mistake. Building list items with `list.innerHTML += '<li>' + input.value + '</li>'`. It opens an XSS hole (the user controls `input.value`) and destroys the listeners on the existing items. Use `createElement + textContent + append`.

Attributes, properties and form values

HTML attributes become properties of the element object (`el.id`, `img.src`, `input.value`, `btn.disabled`); `getAttribute/setAttribute` **work with the attribute text**, and `data-*` **attributes hold your own data in `el.dataset`**.

Attribute What is written in the HTML: the starting configuration. Read and write it with `getAttribute`, `setAttribute`, `removeAttribute` and `hasAttribute`; they work for any attribute, ARIA ones included (see [ARIA and keyboard focus](#)).

Property The live state of the element object now: `el.id`, `img.src`, `input.value`, and boolean ones such as `btn.disabled = true` and `checkbox.checked`.

data-* attribute Your own data: `<li data-task-id="7">` is read as `li.dataset.taskId` (the name turns into camelCase).

Example. For `<a href="/about">`, `link.getAttribute('href')` is `"/about"` (what you wrote) while `link.href` is the full resolved URL such as `"https://example.com/about"`. Same attribute, two views.

Common mistake. Reading `input.value` once when the script starts (it is still empty) and using that stale value in the click handler. Read `input.value` **inside** the handler, at the moment the user acts.

ARIA attributes and keyboard focus

ARIA (Accessible Rich Internet Applications) attributes tell assistive technology what an element is and what state it is in; **keyboard focus** decides which element receives the keys. When JavaScript changes the page, it must keep both right.

<code>aria-label</code>	A name for an element with no visible text: an icon button <code><button aria-label="Delete Buy milk">x</button></code> .
<code>aria-expanded</code>	"true" or "false" on the button that opens a menu or panel. Update it in the same handler that opens it.
<code>aria-live="polite"</code>	A region whose text changes are read out: "Task added".
<code>aria-hidden="true"</code>	Hides decorative content from assistive technology. Never on something focusable.

Keyboard focus

- **Tab order follows the DOM order.** Only interactive elements (links, buttons, form fields) take focus.
- `el.focus()` moves the focus: after adding an item, back to the input; after deleting one, to a sensible neighbour. `document.activeElement` is the element that has it.
- `tabindex="0"` makes another element focusable (prefer a real `<button>`); `tabindex="-1"` makes it focusable by script only.
- **Show the focus:** never remove the outline; style it with `:focus-visible`.

Example. A disclosure button: `btn.setAttribute('aria-expanded', String(!open)); panel.hidden = open;` keeps the announced state in step with what is shown. In the Try it box, Tab to the buttons and press Enter: the console logs the new `aria-expanded` value, and after "Add" the focus is back on the input.

Common mistake. Making a clickable `<div>` with a `click` listener. It cannot be reached with Tab, ignores Enter and Space, and a screen reader does not announce it as a button. Adding `role="button"` does not fix it: ARIA describes an element but adds no behaviour. Use a `<button>`, which is focusable and works with Enter and Space for free.

classList: styling through classes

`el.classList` **adds, removes, toggles and tests CSS classes**, so JavaScript decides **which state** an element is in and CSS decides **how** each state looks.

<code>add('a'), remove('a')</code>	Turn one class on or off, leaving the others.
<code>toggle('a')</code>	Adds it if missing, removes it if present, and returns the new state.
<code>toggle('a', condition)</code>	Forces it on when the condition is true, off when it is false.
<code>contains('a'), replace('a', 'b')</code>	Tests for a class; swaps one for another.

Example. A dark-mode button: `toggleBtn.addEventListener('click', () => document.body.classList.toggle('dark'));` with all the colours defined under `body.dark { ... }` in the CSS.

Common mistake. Writing `el.className = 'done'` to mark a task. `className` is the whole class string, so an element with `class="task urgent"` loses both classes. `el.classList.add('done')` adds one class and keeps the others.

Propagation and delegation

Event propagation: capture, target and bubble

An event does not stay on the clicked element: it travels from the top of the tree down to the target (**capture**),

reaches the target, then climbs back up through every ancestor (**bubble**), running their listeners on the way.



One click visits every ancestor twice: on the way down and on the way up.

- **Plain** `addEventListener(type, fn)` **listens while the event bubbles:** the innermost handler runs first, then each ancestor's, from the inside out.
- `{ capture: true }` **listens on the way down:** an ancestor's capture listener runs before anything at the target.
- `e.eventPhase` says where the event is: 1 capture, 2 at the target, 3 bubble.
- **Most events bubble** (click, input, submit, keydown). `focus` and `blur` do not (`focusin` and `focusout`), nor do `mouseenter` and `mouseleave`.

Example. Click the button in the Try it box: the log shows capture on `#outer` first, then the button's own listener, then bubble on `#middle` and on `#outer`. Click the section padding instead and the button is not on the path at all.

Common mistake. Believing a click belongs only to the element that was clicked. In a to-do list, a click on an item's delete button **also** reaches the item's own click listener (it bubbles), so the item is toggled as it is removed, unless your code checks what was clicked.

stopPropagation, and when not to use it

`e.stopPropagation()` **stops the event from travelling any further, so listeners on the ancestors do not run; it does not cancel the browser's default action.**

Two methods for two different jobs

Method	Ancestors' listeners still run?	Browser default (submit, navigate, tick) still happens?
Neither	Yes	Yes
<code>e.preventDefault()</code>	Yes	No
<code>e.stopPropagation()</code>	No	Yes
Both	No	No

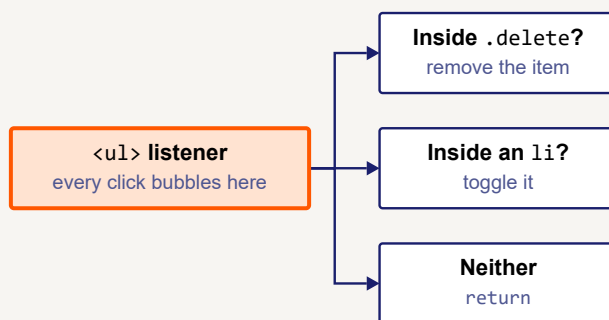
Example. A card that opens a detail view when clicked contains a "favourite" star button. In the star's handler, `e.stopPropagation()` keeps the card from opening. The alternative is in the card's handler: `if (e.target.closest('.star')) return;`

Common mistake. Calling `stopPropagation()` to stop a link from navigating or a form from submitting. It stops the travel, not the default: the page still navigates. That job belongs to `preventDefault()`.

Event delegation

Event delegation means attaching **one** listener to a common ancestor (such as the `<ul>`) and using `e.target.closest(...)` to find which child was hit, instead of one listener per child; it also covers children added

later.



One listener on the list decides what each click means, for current and future items.

1. Listen on a stable ancestor that is in the HTML from the start.
2. Find the relevant element with `e.target.closest(selector)`.
3. If it is null, the click was on nothing you care about: return.
4. Read identifying data from `data-*` attributes.

```

list.addEventListener('click', (e) => {
  const deleteBtn = e.target.closest('.delete');
  if (deleteBtn) {
    deleteBtn.closest('li').remove();
    return; // do not also toggle
  }
  const item = e.target.closest('li');
  if (!item) return; // click on the list padding
  item.classList.toggle('done');
});
  
```

Example. In the Try it box, add three new items with the button, then click and delete them: they work immediately, although no listener was ever attached to them. The only listener is the one on the `<ul>`.

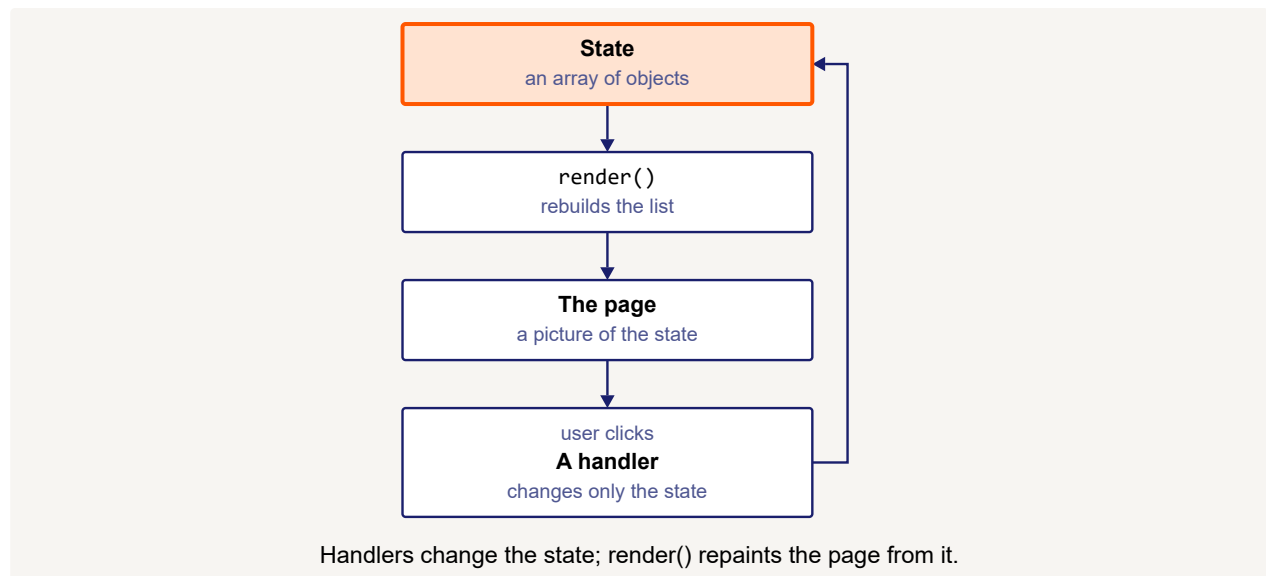
Common mistake. Using `e.target` directly instead of `closest`. If an item is `<li><span>Buy milk</span> ...</li>`, a click on the text gives `e.target = the <span>`, so `e.target.classList.toggle('done')` styles the span, and `e.target.dataset.id` is undefined.

Putting it together

State → render: a small to-do pattern

Keep the data in a JavaScript array (the **state**), let event handlers change only the state, and call one `render()`

function that rebuilds the list from the state after every change.



State An array of objects such as { id: 1, text: 'Buy milk', done: false }.

Handlers Add (push), toggle (find by id, flip done) or remove (filter), then call render(). They never touch the DOM.

render() Empties the list and creates one per item with textContent (safe from XSS), a done class when needed and a data-id that the delegated listener reads.

- **Clean the input:** input.value.trim(), and ignore empty text.
- **Keyboard users:** put the toggle on a real <button> inside the item, not on the (see [ARIA and keyboard focus](#)).
- **In your own page, use a <form>** with a submit listener and preventDefault() (see [preventDefault](#)). The sandboxed Try it box cannot submit forms, so it listens to the button and to the Enter key directly.

```

const todos = [];          // state
let nextId = 1;

function render() {
  list.replaceChildren();    // start from empty
  for (const todo of todos) {
    const li = document.createElement('li');
    li.dataset.id = todo.id;
    li.classList.toggle('done', todo.done);
    const text = document.createElement('button');
    text.className = 'toggle';
    text.textContent = todo.text;          // never innerHTML
    const del = document.createElement('button');
    del.className = 'delete';
    del.textContent = 'x';
    del.setAttribute('aria-label', 'Delete ' + todo.text);
    li.append(text, del);
    list.append(li);
  }
}

```

Example. Adding "Buy milk" pushes { id: 1, text: 'Buy milk', done: false } and renders one item; clicking it flips done to true and renders again (now crossed out); clicking x filters it out of the array and renders an empty list. At every moment todos describes exactly what is on screen.

Common mistake. Updating the DOM and the array separately. Remove the `<li>` but forget to remove the array item, and it reappears the next time `render()` runs. With state $\rightarrow$ render, handlers never touch the DOM directly; they change the state and call `render()`.